PRACTICAL IDE AGENT WORKSHOP

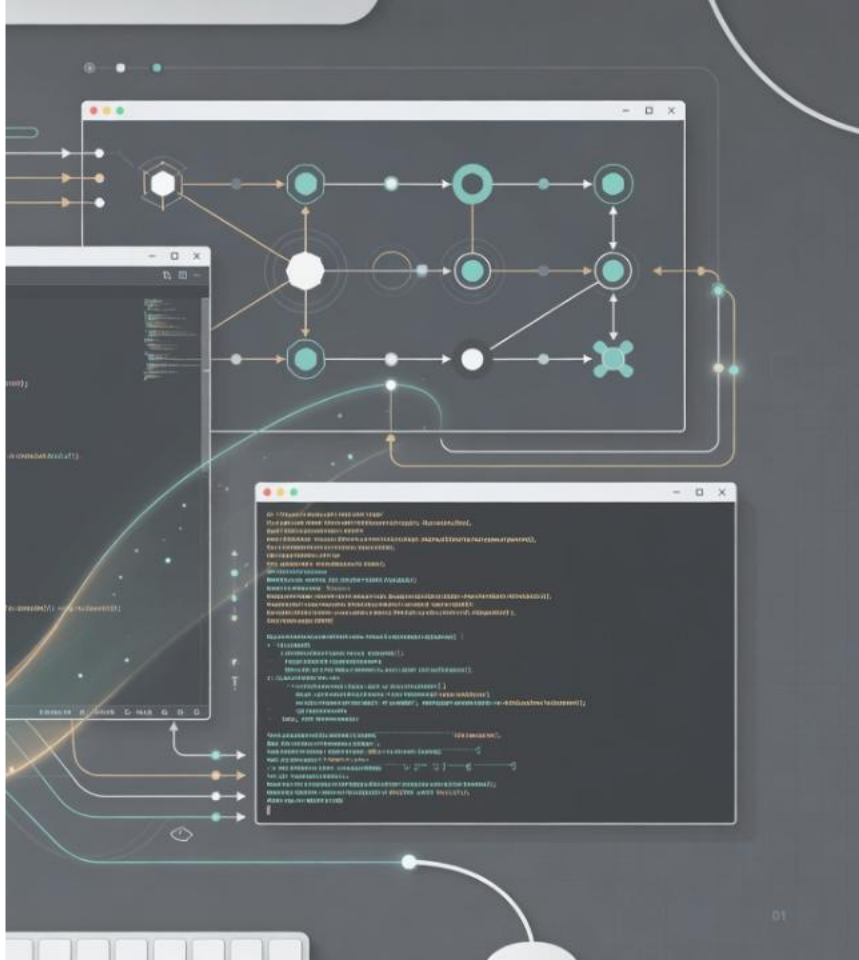
101 AI Basics for Developers

A practical workshop for using IDE agents without losing engineering judgment.

VS Code + GitHub Copilot framing

30 minutes

30 minutes | VS Code + GitHub Copilot mental model



An LLM is a probabilistic text engine wrapped in product constraints.

Tokens

The model sees chunks of text, code, and symbols, not files the way your editor does.

Context

Only the current assembled context is available at generation time.

Uncertainty

Plausible output is not the same as true output, especially for APIs and repo-specific behavior.

Reasoning

Modern models can plan and inspect, but they still need grounding and verification.

Treat model output as a draft from a very fast engineer with incomplete local state.

Good agent prompts look like small implementation briefs.

Weak prompt

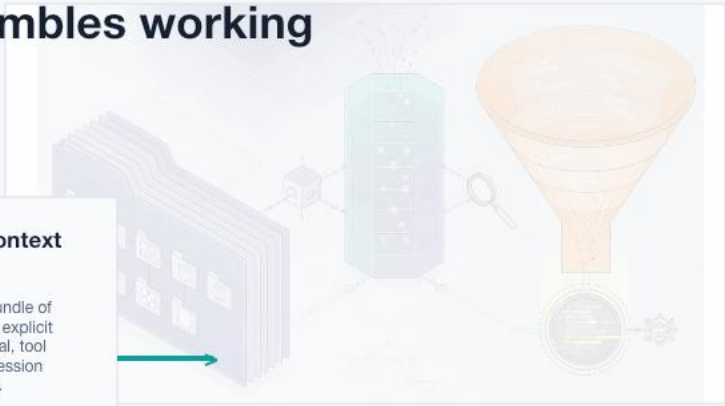
```
Fix auth bug.
```

Agent-ready prompt

```
Investigate why expired sessions still pass /api/me. Start by reading auth
middleware and session tests. Propose the smallest fix, update tests, and
summarize the diff.
```

Prompt quality is context selection plus acceptance criteria. The words matter less than the boundaries.

It does not load the whole repo. It assembles working context.



Repo surface — Files, folders, symbols, tests, docs, issues

Repo index — Semantic code search finds relevant code by meaning

Explicit context — Selected files, prompt text, logs, instructions

Agent tools — Search, read, run commands, edit, inspect diffs

Context window — Only assembled state fits; long sessions compact

Working context

A temporary bundle of retrieved code, explicit prompt material, tool results, and session memory.

Careful mental model

Copilot assembles context. It does not simply paste the whole repository into the model.

GitHub Docs: repo indexing + semantic code search | GitHub Docs: Copilot CLI context management

The most useful skill is choosing what the agent should see first.

Explicit

Selected files, issue text, pasted logs, failing stack traces.

Persistent

.github/copilot-instructions.md, path instructions, AGENTS.md-style guidance.

Retrieved

Semantic code search, repo index, file search, related symbols.

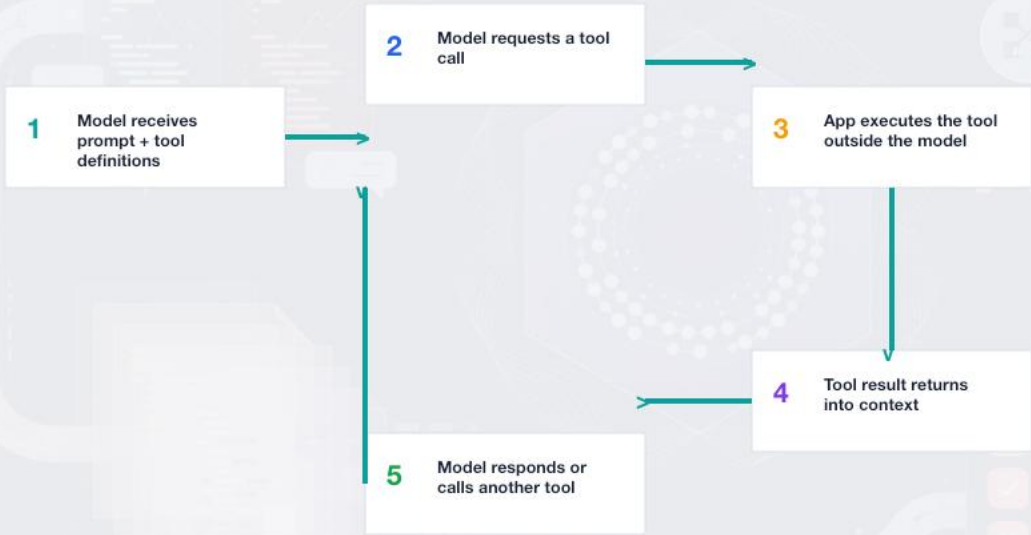
Generated

Tool results, test output, diffs, plans, summaries, checkpoints.

Better inputs reduce guessing. Better checkpoints reduce drift.

GitHub Docs: custom instructions | GitHub Docs: Copilot CLI context management

Agents become useful when the model can ask the environment for facts.

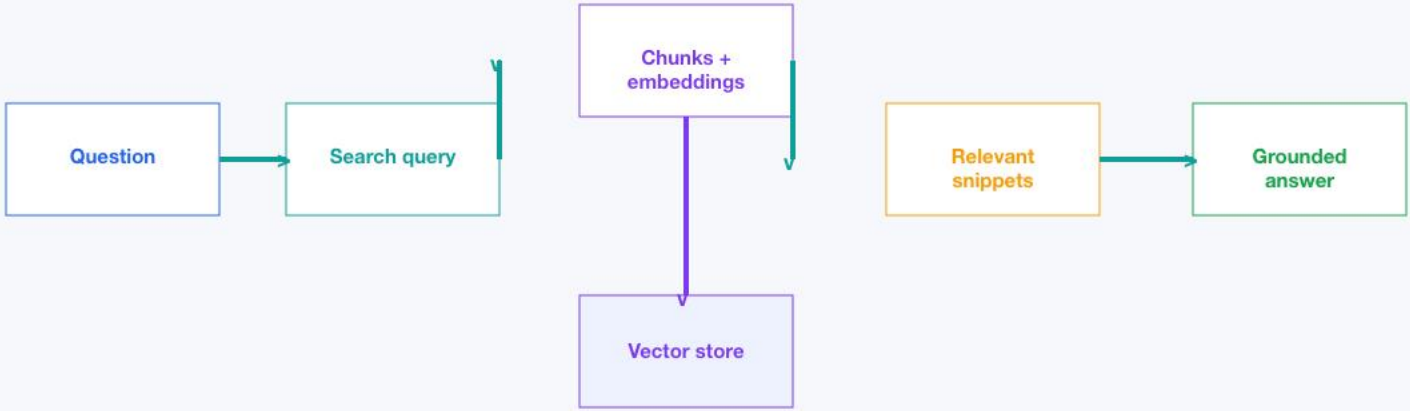


Engineering rule

A tool call is not magic. Your app or IDE executes it, with permissions and consequences.

OpenAI Docs: function/tool calling

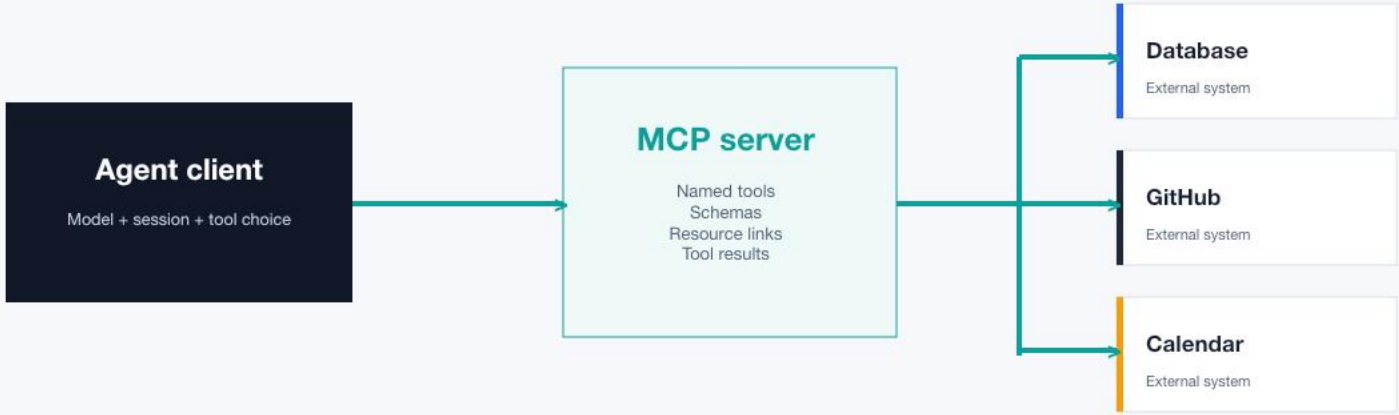
Retrieval is how fresh repo facts get into a bounded context window.



Retrieval helps with facts, but it can still retrieve the wrong fact. Always inspect citations or files for important changes.

OpenAI Docs: retrieval + vector stores

MCP is the adapter layer that lets agents discover and invoke external tools.



The protocol standardizes discovery and invocation. Your security model still has to decide what tools are allowed.

The more dynamic the path, the more you need steering and verification.

Workflow

Predefined code path orchestrates model and tools. Great for repeatable, auditable processes.

Agent

Model dynamically chooses process and tool use. Great for ambiguous tasks that need exploration.

Use workflows when the path is known. Use agents when discovery is part of the job.

IDE AGENT MODES

Choose autonomy based on blast radius, not ambition.

Ask

Explain, compare, locate, summarize. No edits.

Edit

Targeted changes in known files. You control the surface.

Agent

Multi-file work with search, commands, tests, and iteration.

Plan / Autopilot

Plan when scope is uncertain. Autopilot only when the task is bounded and tests are clear.

More autonomy means more need for tests, permissions, and review.

GitHub Docs: Copilot agent sessions

TASK SELECTION

Agents shine on bounded, inspectable work.

Good fit

01	Add a small feature with clear acceptance tests
02	Trace a bug from log to codepath
03	Refactor a repeated pattern with test coverage
04	Generate first-pass tests for existing behavior

Bad fit

01	Rewrite the architecture
02	Make it production ready
03	Fix all flaky tests
04	Change auth and billing in one pass

STEERING LOOP

The best users manage agents like junior teammates with good tools.

Interrupting early is cheaper than reviewing a confident wrong diff.

FAILURE MODES

Most agent failures are context failures wearing a confidence costume.

Hallucinated APIs

Counter: ask it to cite the local symbol or docs before coding.

Overbroad edits

Counter: constrain files, public APIs, and migration scope.

Stale context

Counter: restart, summarize, or checkpoint after major pivots.

Weak tests

Counter: require failing test first or explicit manual QA steps.

Permission risk

Counter: keep dangerous commands and secret access gated.

TEAM SETUP

Make the repo easier for humans and agents to understand.

.github/copilot-instructions.md	Broad repo conventions and engineering preferences.
.github/instructions/*.instructions.md	Path-specific rules for tests, APIs, frontend, infra.
AGENTS.md / CLAUDE.md / GEMINI.md	Agent-facing project workflow notes when supported.
Skills / prompt files	Repeatable workflows such as bug triage, code review, release notes.

This is documentation with operational leverage: humans read it occasionally; agents read it every session.

GitHub Docs: custom instructions

A reliable agent session has three checkpoints.

Before

- [ ] Define done
- [ ] Name files or areas
- [ ] State constraints
- [ ] Pick mode

During

- [ ] Review plan
- [ ] Watch tool calls
- [ ] Interrupt drift
- [ ] Ask for assumptions

After

- [ ] Inspect diff
- [ ] Run tests
- [ ] Check edge cases
- [ ] Capture learnings

CLOSING EXERCISE

Turn a vague request into an agent-ready task.

Vague

Make the settings page better.

Agent-ready

Review the current settings page UX. Identify the smallest accessibility and layout improvements that do not change routes or API behavior. Implement them in the settings component only, add or update focused tests, and summarize before/after behavior.

What is the smallest useful scope?

What should the agent inspect first?

How will we know it worked?

Your job is to keep the agent inside a useful problem shape.